

Thapar Auto

Prototype Document

Product & System Design Prototype
Thapar Institute of Engineering and Technology

September 14, 2026

1 Overview

Thapar Auto is a shared campus auto-rickshaw booking and pooling application built for Thapar Institute of Engineering and Technology. It allows students to book seats on autos plying between campus stops, with the system automatically pooling compatible requests onto autos already en route, or dispatching the next idle auto from a queue when no match exists. Drivers register their vehicle and UPI QR code, join an idle queue, and receive trip assignments through the app.

Technology stack:

- **Frontend:** Flutter (Android, iOS, Web, Windows, Linux targets)
- **Backend:** Node.js + Express REST API
- **Database:** SQLite (via `better-sqlite3`)
- **Auth:** Google Sign-In restricted to `@thapar.edu` accounts (students); Firebase Phone-OTP authentication (drivers)
- **Payments:** UPI QR code scan-and-pay (driver-supplied static QR)

2 Prototype Scope

This prototype implements the end-to-end booking loop: role selection → authentication → stop/seat selection → live availability → booking request → UPI payment → trip confirmation, alongside the symmetric driver flow: registration → idle queue → dispatch/pooling → trip completion. Live GPS tracking of the auto is explicitly out of scope for this iteration.

3 Screen-by-Screen Walkthrough

Screen 1 — Role Selection

Route: `RoleSelectScreen` (app entry point)

Purpose: Lets the user declare whether they are a Student or a Driver before any authentication happens.

Key elements:

- App icon (*directions_car-filled*) and title “Thapar Auto” with tagline “Campus auto booking, made fast.”
- Primary button “I’m a Student” → navigates to the Student Login screen
- Secondary (outlined) button “I’m a Driver” → navigates to the Driver Login screen

Screen 2 — Student Login

Route: StudentLoginScreen

Purpose: Authenticates the student via Google Sign-In and restricts access to institute email addresses.

Flow:

1. Student taps **Sign in with Google**.
2. The returned account email is checked against the @thapar.edu domain; non-institute accounts are signed out immediately with an inline error.
3. On success, the app calls `POST /students` to register/fetch the student record, then stores the session in `AppState` and navigates to the Student Home screen.

Note: the current prototype trusts the client-supplied email; production would additionally verify the Google ID token server-side.

Screen 3 — Student Home (Booking)

Route: StudentHomeScreen

Purpose: Core booking screen — the student selects pickup/drop stops and seat count, watches live pooling availability, and submits a booking request.

Key elements:

- **Pickup** and **Drop** location dropdowns (`LocationDropdown`), populated from `GET /locations`
- **Seat selector** widget (1–capacity)
- Live **availability panel** — debounced call to `GET /availability` on every selection change, showing poolable enroute autos and idle-queue count
- **Request Booking** primary button → `POST /bookings`

Screen 4 — Payment

Route: StudentPaymentScreen

Purpose: Displays the assigned driver's UPI QR code and fare so the student can pay before boarding.

Flow:

1. On load, fetches the assigned auto (`GET /autos/:id`) and its driver (`GET /drivers/:id`) to display the QR image and driver details.
2. Student scans the QR and pays externally, then taps **I've Paid**.
3. App calls `POST /bookings/:id/pay`, flipping the booking's `payment_status` to PAID.
4. Navigates to the Trip Status screen.

Note: this is a scan-and-confirm flow, not a verified payment gateway webhook — flagged in-code for production hardening.

Screen 5 — Trip Status (Confirmation)

Route: StudentTripStatusScreen

Purpose: Confirms the booking and summarizes the trip.

Key elements:

- Success icon and “*You’re booked!*” message
- Contextual message distinguishing a **pooled** booking (joined an already-enroute auto) from a **freshly dispatched** one
- Trip summary card: pickup → drop, seats booked, fare paid, truncated booking ID
- **Book Another Ride** button, returning to Student Home

Screen 6 — Driver Login & Registration

Route: DriverLoginScreen

Purpose: Three-step driver onboarding: phone verification, then vehicle details.

Steps (`_Step enum`):

1. **Phone** — driver enters mobile number; Firebase sends an OTP.
2. **OTP** — 6-digit code entry with a 59-second countdown and resend.
3. **Details** — name, vehicle number, capacity (4/5/7-seater selector), and a required UPI QR code image upload (via `image_picker`).

On submission, calls `POST /drivers` (multipart, with the QR image), which also creates the driver’s **auto** record and places it at the back of the idle queue.

Screen 7 — Driver Dashboard

Route: DriverDashboardScreen

Purpose: Live operating screen for a driver once registered; polls the backend every 5 seconds.

States shown:

- **Idle:** current position in the idle queue (`GET /autos/:id/queue-position`)
- **Enroute:** list of current trip’s bookings (`GET /autos/:id/bookings`), including student name, pickup/drop, seats and payment status
- **Mark Trip Complete** button → `POST /autos/:id/complete-trip`, which resets the auto to idle and re-queues it at the back

4 Navigation Map

From	To
Role Select	Student Login / Driver Login
Student Login	Student Home (on successful auth)
Student Home	Payment (on successful booking request)
Payment	Trip Status (on payment confirmation)
Trip Status	Student Home (Book Another Ride)
Driver Login	Driver Dashboard (on successful registration)
Driver Dashboard	(self-refreshing; no forward navigation)

5 System Design References

The following diagrams accompany this prototype and are included in the delivered package:

- **Use-Case Diagram and Data Flow Diagrams (Levels 0–2)** — see `Thapar Auto System Design Diagram`
- **Entity-Relationship Diagram** — `er_diagram.png` (Figure 1)
- **Activity Diagram** (booking & dispatch flow) — `activity_diagram.jpg` (Figure 2)
- **Project Gantt Chart** — `gantt_chart.pdf`

Thapar Auto — ER Diagram

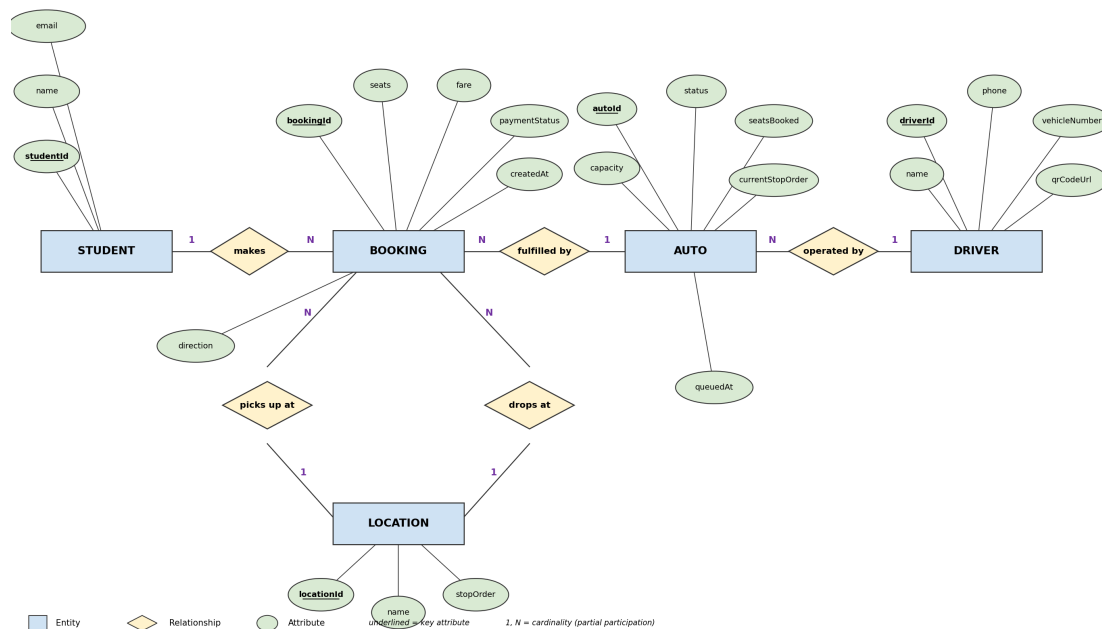


Figure 1: Entity-Relationship Diagram — Thapar Auto

6 Data Model Summary

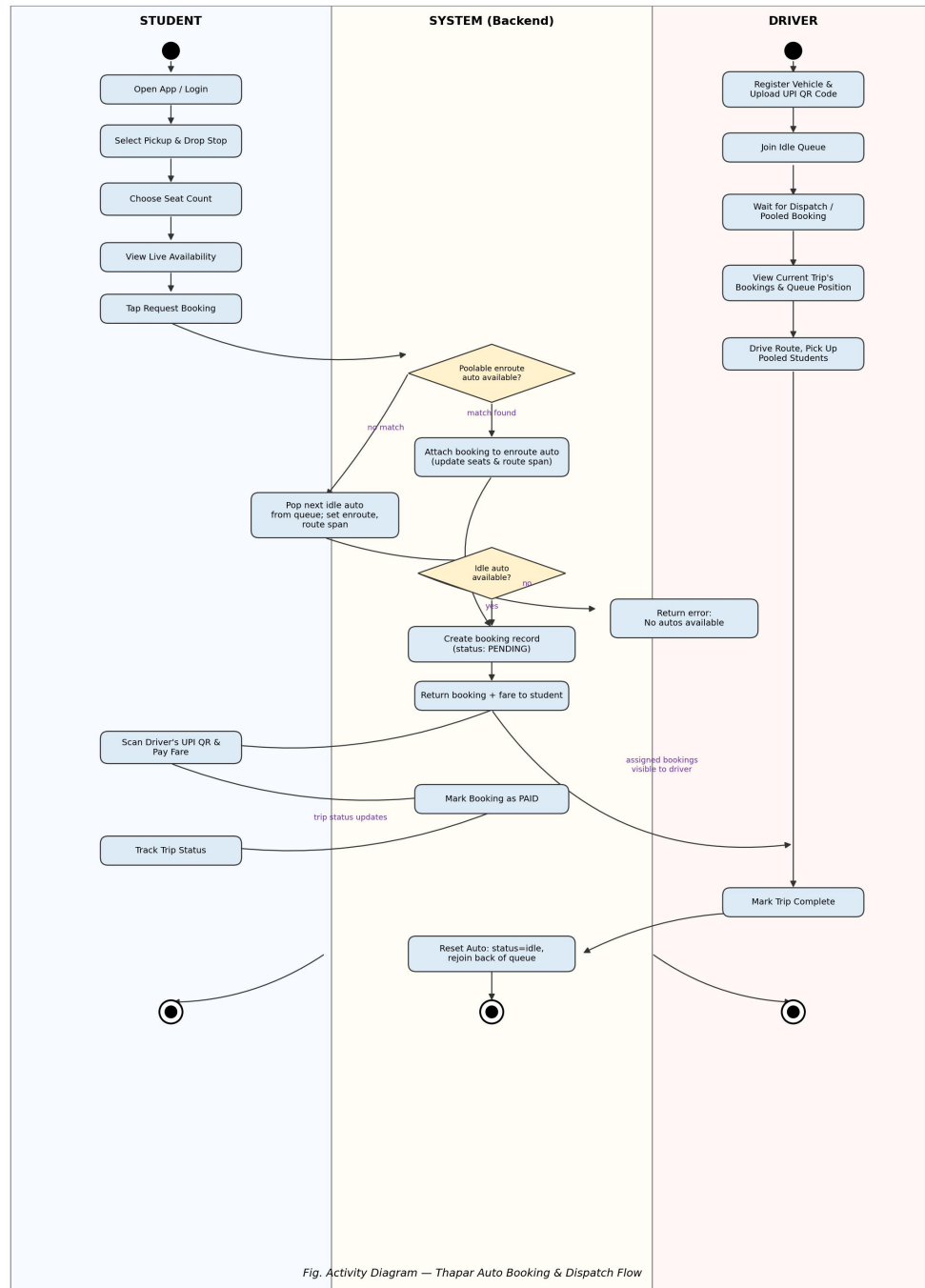


Figure 2: Activity Diagram — Booking & Dispatch Flow

Table	Purpose
locations	Ordered campus stops (<code>stop_order</code> drives routing/direction logic)
students	Student profile, restricted to <code>@thapar.edu</code>
drivers	Driver profile, vehicle capacity, UPI QR image URL
autos	One per driver; tracks <code>status</code> (idle/enroute), <code>seats_booked</code> , direction and route span, and idle-queue timestamp
bookings	Student's seat request: pickup/drop, seats, fare, <code>payment_status</code>

7 Matching Logic (Summary)

On a booking request, the backend first searches for an **enroute** auto heading the same direction whose remaining capacity and route span can absorb the new pickup/drop (pooling). If none qualifies, the **next idle auto** is popped from the FIFO queue and dispatched fresh. Completed trips return the auto to the back of the idle queue. Fare is a flat **Rs. 10 per seat**.